第25章 全點對最短路徑：詳解

25-E01 兩種 DP 的第二步

推導

最多2邊的$L^{(2)}$ rows為[0,4,2,10]、[∞,0,−2,1]、[∞,4,0,3]、[∞,1,−1,0]。允許中介{1,2}的$D^{(2)}$ rows為[0,4,2,10]、[∞,0,−2,6]、[∞,∞,0,3]、[∞,1,−1,0]。

$L^{(2)}[2,4]=1$使用2→3→4，這條路的中介3不在{1,2}，所以$D^{(2)}[2,4]$仍6。$L^{(2)}[3,2]=4$使用3→4→2，中介4也不被$D^{(2)}$允許，因此$D^{(2)}[3,2]=\infty$。

1到4的min-plus四候選為k1:0+∞=∞，k2:4+6=10，k3:7+3=10，k4:∞+0=∞，取10。到3邊可用1→2→3→4，$L^{(3)}[1,4]=4-2+3=5$；以$L^{(2)}[1,3]+W[3,4]=2+3$也得5。

結論與檢核

「第二步」分別限制邊數與中介點，不能交換解讀。n=4無負cycle，$L^{(3)}[1,4]=D^{(4)}[1,4]=5$；不是因兩個DP的第二步相同。

25-E02 負對角與局部失效

推導

2→3→2的總權為−3+1=−2。能進入這圈的起點為{1,2,3}；能從圈走出的終點為{2,3,4}，其笛卡兒積的9個點對皆為−∞。完整rows為[0,−∞,−∞,−∞,∞]、[∞,−∞,−∞,−∞,∞]、[∞,−∞,−∞,−∞,∞]、[∞,∞,∞,0,∞]、[∞,∞,∞,∞,0]。

1雖可進圈，但圈沒有回1的出口，唯一1→1最佳為零長路徑，$\delta(1,1)=0$。1可進圈繞r次後到4，成本$2-3-2r+4=3-2r$，無下界，故$\delta(1,4)=-\infty$。4無出邊，$\delta(4,1)=\infty$；5獨立仍有零長路徑，$\delta(5,5)=0$。

結論與檢核

$D[2,2]<0$是負closed walk的證據，不是最小值；再繞一次還可降低2，所以$\delta(2,2)=-\infty$。有限0、不可達∞與無下界−∞是三種不同語義。

25-E03 Johnson 的成本模型

推導

Floyd為$n^3=10^9$次三索引更新。全零potential在原圖掃n−1輪加檢查，量級$nm=5\times10^6$次邊處理，另O(n)初始化。重複binary-heap Dijkstra量級$n(m+n)\log_2 n\approx1000\cdot6000\cdot9.966\approx5.98\times10^7$；Fibonacci版本為$nm+n^2\log_2 n\approx5\times10^6+9.966\times10^6\approx1.50\times10^7$。前者與後者使用不同priority-queue操作保證，不能混用公式。

第25章 全點對最短路徑：詳解 （續）

完整輸出$n^2=10^6$個距離，若每格佔一word，輸出常駐空間$\Theta(n^2)$；Johnson的工作圖與單source陣列另外$O(n+m)$。若$m=0$，直接建擴增圖仍有n條$s\to v$邊與$n+1$點，一般Bellman–Ford上界$O((n+1)n)$，並非零。全零初始化在原圖無邊時則可直接交付$h=0$，成本$O(n)$。

結論與檢核

Fibonacci-heap總界$O(nm+n^2\log(n+1))$、binary-heap總界$O(n(m+n)\log(n+1))$；無邊也必初始化或寫出資料。以上項數比較不代表真實秒數或加速比例。

25-E04 Potential 與距離還原

推導

初始$h=(0,0,0,0)$。第一輪$2\to3$權-2使$h[3]=-2$，其餘零；第二、三輪不再改變。每條$\hat{w}=w+h[u]-h[v]$依題序為$1\to2$:4、$1\to3$:9、$2\to3$:0、$2\to4$:6、$3\to4$:1、$4\to2$:1。

從1初始$(0,\infty,\infty,\infty)$。取1後$(0,4,9,\infty)$；取2後3候選$4+0=4<9$，4候選$4+6=10$，成$(0,4,4,10)$；取3後4候選$4+1=5<10$，成$(0,4,4,5)$；取4後2候選$5+1=6\geq4$，不變。

還原$\delta(u,v)=\delta\text{Hat}(u,v)-h[u]+h[v]$：source1因$h[1]=0$，得$(0,4,4-2,5)=(0,4,2,5)$。source3的$\delta\text{Hat}(3,2)=2$，原距離$2-(-2)+0=4$；source4到3的$\delta\text{Hat}=1$，原距離$1-0-2=-1$。

結論與檢核

最短$1\to4$為$1\to2\to3\to4$，原權$4-2+3=5$，重加權$4+0+1=5$。potential起終差對此pair為0，但對$1\to3$為+2，不能因此誤以為重加權數值總相同。

25-E05 重加權不等式的兩方向

推導

若存在有限h使每條$\hat{w}\geq0$，沿任意cycle相加，所有h望遠鏡消去，得到cycle原權$=\Sigma\hat{w}\geq0$，所以不能有負cycle。這證明必要性，並不需要h真的是某個source的距離。

若無負cycle，加入只向外的super-source與零權邊，新圖亦無負cycle；每點可達且最短距離可取simple，故$h(v)=\delta(s,v)$有限。把一條最短$s\to u$接$u\to v$，得到$h(v)\leq h(u)+w(u,v)$，即$\hat{w}(u,v)\geq0$，證明充分性。

反例取$1\to4$權6、$1\to2$權2、$2\to4$權2。原兩段成本4勝直達6；每邊加3，直達成本9、兩段10，次序反轉。potential法中每條$u\to v$增加$h[u]-h[v]$，整條路只加$h[1]-h[4]$，兩條路相同，故次序不變。

結論與檢核

有限非負重加權potential存在$\Leftrightarrow$無負cycle。保序依賴端點差可望遠鏡消去，不能用「所有邊都加大了」替代這個條件。

第25章 全點對最短路徑：詳解 （續）

25-E06 零權 Cycle 的前驅

推導

source1的最佳(cost,hops)為1:(0,0)、2:(0,1)、3:(1,1)。到3直接邊1→3是(1,1)，經2則(1,2)，字典序選直接。可設parent[2]=1、parent[3]=1，parent[1]=NIL。回溯3←1，恰1邊成本1。

只看cost tight，1→2與2→1皆保持距離0，因此若錯設parent[1]=2、parent[2]=1就形成cycle；source無parent的規格已被違反。即使只保留非source任選，還可造source s分別以權0到兩點u,v，加u↔v權0，選parent[u]=v、parent[v]=u，也會形成cycle；所以不能只靠source置NIL就宣稱所有任選緊邊都安全。hops條件要求parent的hop少1，直接排除這種圈。

對單邊1→2權1，初始Π[1,2]=1。k=2的候選D[1,2]+D[2,2]=1+0與舊值相同；若等值也更新，就設Π[1,2]=Π[2,2]=NIL，反而失去路徑。

結論與檢核

最少hop的tie rule給明確下降量，讓前驅無環與終止可證。一般Floyd前驅採嚴格改善、等值保留；不要把「存在最短路徑」誤當任意指標選擇皆正確。

25-E07 結合律與 Loop Order

推導

$((A \otimes B) \otimes C)[i,j] = \min_q(\min_p(A[i,p] + B[p,q]) + C[q,j]) = \min_{p,q}(A[i,p] + B[p,q] + C[q,j])$。改先對q取min，得到$\min_p(A[i,p] + \min_q(B[p,q] + C[q,j])) = (A \otimes (B \otimes C))[i,j]$。有限索引與可分配的min/加法允許重排。

n=6無負cycle時有最短simple path，最多5邊。W→W^2→W^4→W^8依序預算1,2,4,8，8≥5；增加預算不會產生更便宜的負圈walk，因此W^8=W^5即最短矩陣。若有負cycle，這個最後等式失去前提，固定預算變大可能繼續下降。

鏈1→2→4→3：在i=1,j=3時，k=2需D[2,3]，尚∞；k=4需D[1,4]，也尚∞，故此格不變。到j=4才用k=2算D[1,4]=2，但不會回頭重算j=3，漏掉長3路徑。正確k外層保證本輪所有輸入都已有前k−1中介的完整能力；且無負cycle令row/column k不變，才可原地更新。

結論與檢核

結合律支持對min-plus「完整矩陣product」分組，不支持任意改單次Floyd的loop order。兩個正確性證明管理的是不同狀態，不能混為同一條代數規則。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第25章 全點對最短路徑：詳解 （續）

原創驗算程式附錄: all_pairs_checks.py

```
"""APSP arithmetic and witnesses checked against exhaustive simple paths/cycles."""
from math import inf
from itertools import product
from heapq import heappush,heappop
import random

SAMPLE={0:[(1,4),(2,7)],1:[(2,-2),(3,6)],2:[(3,3)],3:[(1,1)]}


def matrix(graph):
    n=len(graph);w=[[0 if i==j else inf for j in range(n)] for i in range(n)]
    for u,neighbors in graph.items():
        for v,cost in neighbors:w[u][v]=min(w[u][v],cost)
    return w


def minplus(a,b):
    n=len(a)
    return [[min((a[i][k]+b[k][j] for k in range(n)),default=inf)
             for j in range(n)] for i in range(n)]


def edge_budget(graph):
    n=len(graph);w=matrix(graph);out=[[[0 if i==j else inf for j in range(n)] for i in
range(n)]]
    for _ in range(max(0,n-1)):out.append(minplus(out[-1],w))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第25章 全點對最短路徑：詳解 （續）

```
    return out


def squaring(graph):
    n=len(graph);a=matrix(graph);budget=1
    while budget<n-1:a=minplus(a,a);budget*=2
    return a


def floyd(graph,inplace=True):
    n=len(graph);d=matrix(graph);stages=[[row[:] for row in d]]
    for k in range(n):
        old=d if inplace else [row[:] for row in d]
        for i in range(n):
            for j in range(n):d[i][j]=min(old[i][j],old[i][k]+old[k][j])
        stages.append([row[:] for row in d])
    return d,stages


def closure(graph):
    n=len(graph);reach=[[i==j or any(v==j for v,_ in graph[i]) for j in range(n)] for i in
range(n)]
    for k in range(n):
        for i in range(n):
            for j in range(n):reach[i][j]=reach[i][j] or (reach[i][k] and reach[k][j])
    return reach
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第25章 全點對最短路徑：詳解 （續）

```
def classify(graph,d):
    n=len(graph);reach=closure(graph)
    return [[-inf if any(d[k][k]<0 and reach[i][k] and reach[k][j] for k in range(n))
             else d[i][j] for j in range(n)] for i in range(n)]


def lex_floyd(graph):
    n=len(graph);d=[[(0,0) if i==j else (inf,inf) for j in range(n)] for i in range(n)]
    for u,neighbors in graph.items():
        for v,cost in neighbors:d[u][v]=min(d[u][v],(cost,1))
    for k in range(n):
        for i in range(n):
            for j in range(n):
                candidate=(d[i][k][0]+d[k][j][0],d[i][k][1]+d[k][j][1])
                d[i][j]=min(d[i][j],candidate)
    return d


def predecessor_from_lex(graph,values):
    n=len(graph);parents=[[None]*n for _ in range(n)]
    for source in range(n):
        for u,neighbors in graph.items():
            for v,cost in neighbors:
                if v!=source and values[source][u][0]<inf and values[source][v]==(
                    values[source][u][0]+cost,values[source][u][1]+1):
                    parents[source][v]=u
    return parents


def johnson(graph):
    n=len(graph);h=[0]*n;trace=[h[:]]
    for _ in range(max(0,n-1)):
        old=h[:]
        for u,neighbors in graph.items():
            for v,cost in neighbors:h[v]=min(h[v],old[u]+cost)
        trace.append(h[:])
    if any(h[v]>h[u]+cost for u in graph for v,cost in graph[u]):return None,h,trace
    adjusted={u:[(v,cost+h[u]-h[v]) for v,cost in neighbors] for u,neighbors in
graph.items()}
    assert all(cost>=0 for neighbors in adjusted.values() for _,cost in neighbors)
    result=[];dijkstra_traces=[]
    for source in range(n):
        d=[inf]*n;d[source]=0;heap=[(0,source)];settled=[]
        while heap:
            cost,u=heappop(heap)
            if cost!=d[u]:continue
            settled.append((u,cost))
            for v,weight in adjusted[u]:
```

第25章 全點對最短路徑：詳解 （續）

```
                if cost+weight<d[v]:
                    d[v]=cost+weight;heappush(heap,(d[v],v))
        result.append([value-h[source]+h[v] if value<inf else inf for v,value in
enumerate(d)])
        dijkstra_traces.append(settled)
    return result,h,trace,dijkstra_traces


def oracle(graph):
    # Enumerate simple directed cycles and simple paths, no dynamic-programming state.
    n=len(graph);cycle_vertices=set()
    for start in range(n):
        def cycles(u,seen,cost):
            for v,weight in graph[u]:
                if v==start:
                    if cost+weight<0:cycle_vertices.update(seen)
                elif v not in seen:cycles(v,seen|{v},cost+weight)
        cycles(start,{start},0)
    reachable=[[False]*n for _ in range(n)]
    answer=[[inf]*n for _ in range(n)];hops=[[inf]*n for _ in range(n)]
    for start in range(n):
        def paths(u,seen,cost,length):
            reachable[start][u]=True
            if (cost,length)<(answer[start][u],hops[start][u]):
                answer[start][u]=cost;hops[start][u]=length
            for v,weight in graph[u]:
                if v not in seen:paths(v,seen|{v},cost+weight,length+1)
        paths(start,{start},0,0)
    for i in range(n):
        for j in range(n):
            if any(reachable[i][k] and reachable[k][j] for k in cycle_vertices):answer[i]
[j]=-inf
    return answer,hops,reachable


def verify(graph):
    answer,hops,reachable=oracle(graph);raw,stages=floyd(graph)
    assert classify(graph,raw)==answer
    buffered,_=floyd(graph,False);assert classify(graph,buffered)==answer
    assert closure(graph)==reachable
    result=johnson(graph)
    negative=any(answer[i][i]==-inf for i in graph)
    assert (result[0] is None)==negative
    if not negative:
        assert raw==buffered==answer==edge_budget(graph)[-1]==squaring(graph)==result[0]
        values=lex_floyd(graph)
        assert values==[[(answer[i][j],hops[i][j]) for j in graph] for i in graph]
        parents=predecessor_from_lex(graph,values)
        for source in graph:
            for target in graph:
                if answer[source][target]==inf:assert parents[source][target] is
None;continue
```

第25章 全點對最短路徑：詳解 （續）

```
                v=target;cost=0;path=[v]
                while v!=source:
                    u=parents[source][v];assert u is not None
                    cost+=next(weight for w,weight in graph[u] if w==v)
                    v=u;path.append(v);assert len(path)<=len(graph)
                assert cost==answer[source][target] and len(path)-1==hops[source][target]
    return not negative


def main():
    total=0;safe=0
    for n in range(4):
        edges=[(u,v) for u in range(n) for v in range(n) if u!=v]
        for weights in product([None,-2,0,3],repeat=len(edges)):
            graph={u:[] for u in range(n)}
            for (u,v),weight in zip(edges,weights):
                if weight is not None:graph[u].append((v,weight))
            safe+=verify(graph);total+=1
    rng=random.Random(25)
    for _ in range(1200):
        n=rng.randrange(1,7)
        graph={u:[(v,rng.randrange(-4,7)) for v in range(n) if rng.random()<.28] for u in
range(n)}
        safe+=verify(graph);total+=1
    verify(SAMPLE)
    expected=[[0,4,2,5],[inf,0,-2,1],[inf,4,0,3],[inf,1,-1,0]]
    assert floyd(SAMPLE)[0]==expected
    assert edge_budget(SAMPLE)[2]==[[0,4,2,10],[inf,0,-2,1],[inf,4,0,3],[inf,1,-1,0]]
    j=johnson(SAMPLE);assert j[1]==[0,0,-2,0]
    assert j[3][0]==[(0,0),(1,4),(2,4),(3,5)]
    # Wrong i,j,k order fails on this 1->2->4->3 chain, even without negative edges.
    chain={0:[(1,1)],1:[(3,1)],2:[],3:[(2,1)]};wrong=matrix(chain)
    for i in range(4):
        for j in range(4):
            for k in range(4):wrong[i][j]=min(wrong[i][j],wrong[i][k]+wrong[k][j])
    assert wrong[0][2]==inf and floyd(chain)[0][0][2]==3
    isolated_negative={0:[(1,2)],1:[(2,-3)],2:[(1,1),(3,4)],3:[],4:[]}
    assert classify(isolated_negative,floyd(isolated_negative)[0])==[
        [0,-inf,-inf,-inf,inf],[inf,-inf,-inf,-inf,inf],
        [inf,-inf,-inf,-inf,inf],[inf,inf,inf,0,inf],[inf,inf,inf,inf,0]]
    zero_cycle={0:[(1,0),(2,1)],1:[(0,0),(2,1)],2:[]};verify(zero_cycle)
    print(f'PASS APSP_weighted_graphs={total} without_negative_cycles={safe} '
          'Floyd_inplace_and_buffered=minplus=Johnson=independent_simple_paths '
          'negative_pairs=enumerated_simple_cycles lex_predecessors_decrease_hops=True '
          'wrong_ijk_counterexample=True sample_potential=0,0,-2,0 sample_restored_1to4=5')


if __name__=='__main__':main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第25章 全點對最短路徑：詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study